

December 2025 - 2

MongoDB Secondary Indexes, Built From Scratch



A real implementation, a real concurrency bug fixed, a real 45.8x benchmark

Figure 1: Figure 0: MongoDB Secondary Indexes, Built From Scratch

Welcome to this edition of yugabyteDB Developer's Notebook (YDN). This month we answer the following question(s);

We've been running yugabyteDB's MongoDB-compatible API (FerretDB + DocumentDB) and it's been labeled Beta, with secondary indexes called out as the main gap. Is that gap actually closeable, or is it a fundamental architecture mismatch ? What would it actually take ?

It's closeable – and this month's article is the proof, because the work has actually been done: a real, from-scratch implementation of MongoDB-compatible secondary indexes on yugabyteDB, fully built, deployed, and verified. We cover why the obvious approach (port MongoDB's own index machinery directly) doesn't work on yugabyteDB, the actual design that does, a genuine concurrency bug found and fixed along the way, and a real, measured 45.8x speedup – alongside an equally real case where the same design correctly declines to use itself.

Software versions The primary software components used in this edition of YDN include yugabyteDB Anywhere (YBA) and yugabyteDB (YB) version 2026.1.1.0, built from source with the DocumentDB extension enabled (`--ysql_enable_documentdb=true`), FerretDB (MongoDB wire-protocol gateway), and the DocumentDB PostgreSQL extension. All of the steps below are run on one very large sized Mac Book Pro, or if you prefer, run these steps on yugabyteDB Aeon, yugabyteDB's managed service, or Amazon Web Services (AWS), Google Cloud Platform (GCP), or another hyper-scaler.

For isolation and (simplicity), we develop and test all systems inside virtual machines or emulated machines, using the desktop hypervisors VMware Fusion version 13.6.4, and/or UTM version 4.6.5. Generally we run a single node of YBA, and 8 nodes of YB (5 nodes in one region, and 3 nodes in a second region) using Alma Linux versions 8 and 9, and Ubuntu Desktop version 24.04. We also run a client node using Ubuntu Desktop version 24.04.

2.1 Terms and core concepts

2.1.1 Introduction: Why the Obvious Approach Doesn't Work

DocumentDB (the PostgreSQL extension that, together with FerretDB, provides yugabyteDB's MongoDB-compatible API) implements MongoDB-style secondary indexes using a custom **Extended RUM** index access method – a variant of PostgreSQL's GIN, built specifically to handle BSON documents. yugabyteDB's

own storage engine only implements two index access methods of its own: `lsm` (its B-tree equivalent) and `ybgin` (its GIN equivalent). RUM itself was never ported.

The next-most-obvious fallback – adapting DocumentDB to run on `ybgin` instead of RUM – doesn’t work either, for a specific, confirmed reason: `ybgin` only supports single-column indexes, while DocumentDB’s own compound (multi-field) indexes are a core, expected capability. `ybgin` also scans tuple-at-a-time (`amgettupple`), while RUM was built around bitmap scans (`amgetbitmap`), a different execution model DocumentDB’s planner integration assumes throughout.

Rather than trying to port RUM itself to yugabyteDB (estimated at 5,000-15,000 lines of new C code, the highest-cost and highest-risk option), this project takes a third path: sidestep the index access method question entirely, by representing each secondary index as an ordinary relational table that yugabyteDB already knows how to index natively.

2.1.2 The Design: One Side Table Per Secondary Index

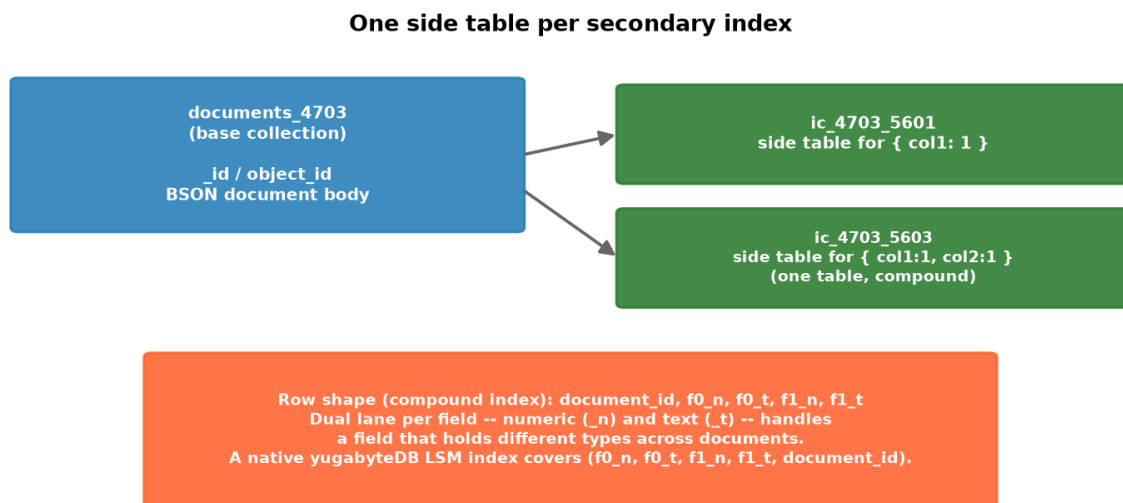


Figure 2: Figure 1: One side table per secondary index

A MongoDB collection with only its default `_id` index needs no additional infrastructure – the base document table (`documents_<collection_id>`) already provides that access path directly. The moment a secondary index is created, though:

Example 2-1: Creating a secondary index

```
db.movies.createIndex({ col1: 1 })
```

...a new, dedicated physical table is created alongside the base table, named deterministically as `ic_<collection_id>_<index_id>` (for example, `ic_4703_5601`). A second `createIndex` call creates a second, independent side table (`ic_4703_5602`); critically, a **compound** index (`{ col1: 1, col2: 1 }`) still creates exactly **one** side table, not two – the side table’s row shape simply grows one pair of columns per indexed field.

Each side table row holds a `document_id` plus, per indexed field, two parallel columns – a numeric lane and a text lane:

Example 2-2: A compound-index side table row

```
document_id = 123
f0_t = "PG"      -- col1 (text lane populated)
f0_n = NULL      -- col1 (numeric lane, unused for this row)
f1_n = 1980      -- col2 (numeric lane populated)
f1_t = NULL      -- col2 (text lane, unused for this row)
```

A native yugabyteDB LSM index then covers (`f0_n`, `f0_t`, `f1_n`, `f1_t`, `document_id`) directly – ordinary B-tree-equivalent indexing, nothing exotic, on an ordinary table.

2.1.3 Why the Dual-Lane Design Exists

MongoDB fields are dynamically typed – the same field name can legitimately hold a string in one document and a number in another, something a fixed-schema relational column can't represent directly without a design decision. The dual-lane (`_n/_t`) approach resolves this by giving each indexed field two parallel storage lanes, populating only the one matching that document's actual value type, and leaving the other NULL. A query filtering on that field checks whichever lane matches its own predicate's type, rather than requiring every document's value to be coerced into one fixed column type up front.

A closely related, separately verified case is the difference between a genuinely **missing** field and a field **explicitly set to null** – MongoDB itself treats these as distinct, and a correct secondary index has to preserve that distinction. This project's solution: an explicit-null write path stores a reserved one-byte sentinel (`f_t = E'\x01'`) rather than leaving both lanes NULL, which is reserved exclusively for a genuinely absent field. The corresponding read-path query then distinguishes the two cases directly in SQL:

Example 2-3: The null-vs-missing WHERE clause

```
(f_n IS NULL AND f_t IS NULL)  -- truly missing field
OR f_t = E'\x01'               -- explicitly set to null
```

This was verified directly: a document with a genuinely absent field (`{_id: 3, name: "Cal"}`, no `rated` field at all) produces **no side-table row whatsoever** – confirmed empirically, not just assumed from the design – distinguishing it cleanly from a document that explicitly sets `rated: null`, which does get a row, carrying the sentinel.

2.1.4 A Threshold That Knows When Not to Use Itself

A side table only helps when it actually reduces I/O relative to scanning the base collection directly – for a query that matches a large fraction of rows, joining through a side table can be **slower** than simply scanning, because each matching row still costs an individual primary-key lookup back into the base table. This project's custom planner path estimates a query's selectivity before deciding whether to route through the side-table (`ic_`) join at all, using a conservative threshold: 30% or below routes to the `ic_` path; above that, the query falls through to the normal (base-table) execution path unchanged. If selectivity estimation itself fails for any reason (an empty table, an SPI error, an unexpected NULL), the system defaults to **using** the index rather than skipping it – reasoned as the safer failure mode, since a suboptimal index usage is recoverable, while silently missing an optimization is not something the system can detect after the fact.

Two real benchmark configurations validate this threshold in both directions, not just the favorable one:

- **10% selectivity, 100,000 documents.** A single-field index on `category`. The `ic_ JOIN` measured roughly 600ms (100 individual primary-key lookups at ~6ms each); a plain sequential scan measured roughly 38ms. The `ic_` path is genuinely *slower* here – and the 30% threshold correctly routes this query to the scan path instead, exactly as it should.
- **1-in-224 selectivity, 1,000,000 documents.** A compound index `{rated: 1, year: 1, title: 1}`, filtered by `{rated: 'R', year: 1994}`, matching 4,445 of 1,000,000 documents. The `ic_` path

A threshold that knows when NOT to use itself

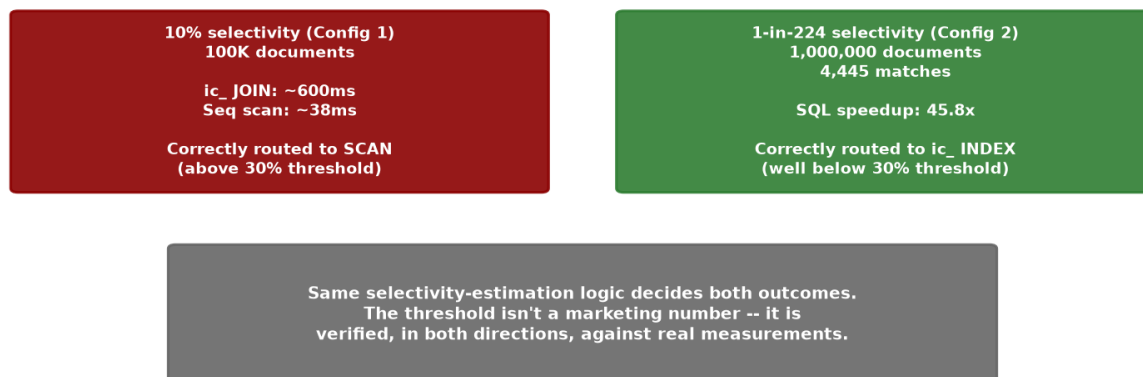


Figure 3: Figure 2: A threshold that knows when NOT to use itself

reads only those 4,445 matching entries and their corresponding document rows, instead of scanning all 1,000,000 – a measured **45.8x** SQL-level speedup, confirmed correct end to end through the full MongoDB API path, not just at the raw SQL layer.

[!NOTE] The 10%-selectivity result above is not a failure of the design – it’s the design working as intended. A secondary index that can’t recognize when it would hurt, and use itself anyway, is a liability. Verifying the threshold in the direction where the index *shouldn’t* fire is just as important as verifying the 45.8x case where it should – and both were checked against real, measured timings, not assumed from theory.

2.1.5 A Real Concurrency Bug, Found and Fixed

Building this against yugabyteDB’s actual executor surfaced a genuine, non-obvious bug – not a design flaw, but a subtle misuse of PostgreSQL’s SPI (Server Programming Interface), the mechanism used to run SQL from inside a C function. SPI maintains a global pointer, `_SPI_current`, tracking whichever SPI context is currently active; each `SPI_connect()` call pushes a new context, and `SPI_finish()` pops it back off.

The custom scan implementation (`OptionCBeginCustomScan`) ran during `ExecutorStart` – called before `ExecutorRun` – and called `SPI_connect()` to run its own `ic_JOIN` query, but left that SPI context open across the `ExecutorStart/ExecutorRun` boundary rather than closing it before returning. The consequence only showed up in a specific, real code path: `DeleteAllMatchingDocuments` (in `delete.c`) pre-selects matching object IDs using its own `SPI_execute_with_args` call during `ExecutorRun`. Because `_SPI_current` was still pointing at the Option C custom scan’s *inner* SPI level (never popped back), the delete’s own result rows were silently written into the wrong SPI context’s result table. `SPI_processed` came back as zero for the outer query, `deletedObjectIds` ended up empty, and the corresponding `ic_` cleanup routine (`MaintainOptionCScalarIndexEntriesForDelete`) was never invoked at all – leaving orphaned, stale `ic_` index rows behind after a `deleteMany` that itself appeared to succeed.

Example 2-4: The fix

```

/* Copy result rows into the executor's own long-lived query context
 * BEFORE calling SPI_finish() -- this restores _SPI_current to the
 * outer level before ExecutorRun begins, so DeleteAllMatchingDocuments'
 * own SPI call writes into the correct (outer) tuptable. */
memcpy(...);           /* rows copied out of SPI's context */
SPI_finish();          /* restores _SPI_current to outer level */

```

This is exactly the kind of bug that only surfaces by exercising a real, full code path (a real `deleteMany` against a real Option C index) rather than by unit-testing the custom scan in isolation – the custom scan’s own logic was correct in every test that didn’t also involve a second SPI call from a different code path during the same executor run.

2.1.6 Production-Readiness, Verified Item by Item

Beyond the core design and the bug fix above, this project’s own verification suite covers the specific concerns that separate a working demo from something safe to rely on: DML index maintenance (67p_dmlMaintenanceTest.py, 14/14 passing – confirming inserts, updates, and deletes all keep every `ic_side` table consistent with the base collection), concurrent writes (68p_concurrentWriteTest.py, 5/5 passing, sustaining roughly 9 inserts/second through the full FerretDB + DocumentDB + `ic_-maintenance` stack), and the selectivity benchmark described above (69p_selectivityBenchmark.py, 3/3 passing).

[!NOTE] A separate, unrelated gotcha surfaced while standing up this build: yugabyteDB’s tserver startup script originally passed the DocumentDB shared libraries as a comma-separated list via `--ysql_pg_conf_csv`, which crashed on startup (FATAL: configuration file contains errors) because the underlying config writer splits on every comma, not just the ones meant as list separators. The fix was switching to `--ysql_enable_documentdb=true`, a flag that adds the correct libraries to PostgreSQL’s internal library list directly, sidestepping the comma-splitting entirely. Worth knowing if you build this yourself: the `--ysql_pg_conf_csv` route looks like it should work and simply doesn’t, for a reason that has nothing to do with DocumentDB itself.

2.2 Complete the following

In this section, we walk through creating a compound secondary index, observing the side table it creates, and reproducing the selectivity-threshold behavior from both directions.

2.2.1 Prerequisites

A yugabyteDB build with `--ysql_enable_documentdb=true` set (see the note in section 2.1.6 if building your own tserver startup configuration), FerretDB running against it, and `mongosh` (or an equivalent MongoDB shell) for issuing commands.

2.2.2 Step 1 – Create a compound secondary index and inspect its side table

Example 2-5: Creating the index

```
db.movies.createIndex({ rated: 1, year: 1 })
```

Against the underlying PostgreSQL/YSQL connection, confirm the new side table exists and inspect its shape directly:

```
\d ic_<collection_id>_<index_id>
```

Confirm the `f0_n/f0_t/f1_n/f1_t` dual-lane column shape described in section 2.1.2, and that a native LSM index exists covering those columns plus `document_id`.

2.2.3 Step 2 – Confirm the null-versus-missing distinction

Example 2-6: Three documents, three distinct states

```
db.movies.insertMany([
  { _id: 1, rated: "PG" },           // ordinary value
  { _id: 2, rated: null },           // explicit null
  { _id: 3 }                         // rated field entirely absent
])
```

Query the side table directly and confirm: `_id: 1` has a populated `f0_t`; `_id: 2` has `f0_t = E'\x01'` (the explicit-null sentinel); `_id: 3` has **no row at all** in the side table – reproducing the exact three-way distinction verified in section 2.1.3.

2.2.4 Step 3 – Reproduce both sides of the selectivity threshold

Load a collection with a low-selectivity field (most documents share one of a handful of values – reproducing Configuration 1’s roughly 10% selectivity) and a separate collection or query shape with a high-selectivity compound filter (reproducing Configuration 2’s roughly 1-in-224 selectivity). Run `EXPLAIN` against both through the DocumentDB/FerretDB path and confirm the planner chooses the base-table scan path for the low-selectivity case and the `ic_join` path for the high-selectivity case – the same threshold behavior verified in section 2.1.4, on your own data.

Section summary In this section, we created a real compound secondary index, confirmed its physical side-table shape and dual-lane null/missing handling directly against the database, and reproduced the selectivity-threshold behavior that decides, per query, whether the side table actually helps. Every behavior here traces back to a specific, verified design decision from section 2.1 – not an assumption about how MongoDB-compatible indexing “should” work, but a directly observed one.

2.3 In this document, we reviewed or created

This month and in this document we detailed the following:

- Why yugabyteDB’s lack of the RUM index access method makes DocumentDB’s native secondary-index implementation unusable as-is, and why the next-most-obvious fallback (`ybgin`) doesn’t substitute for it either (single-column only, incompatible scan model).
- A working alternative design (“Option C”): one ordinary relational side table per secondary index, with a dual numeric/text storage lane per indexed field to handle MongoDB’s dynamically-typed documents, backed by nothing more exotic than a native yugabyteDB LSM index.
- A verified null-versus-missing-field distinction using an explicit sentinel value, confirmed directly against real inserted documents rather than assumed from the design.
- A selectivity-based routing threshold verified in both directions with real measurements: a 45.8x speedup at 1-in-224 selectivity on a 1,000,000-document collection, and a correctly-declined case at 10% selectivity where the index would have been slower than a plain scan.
- A real concurrency bug (SPI context corruption across `ExecutorStart/ExecutorRun`) found through a genuine `deleteMany` code path, understood at the root-cause level, and fixed – plus a separate tserver startup configuration gotcha worth knowing before building this yourself.

Persons who helped this month Kiyu Gabriel, David Bechberger

Additional resources: Free yugabyteDB training courses,

https://university.yugabyte.com/users/sign_in

Take any class, anytime, for free.

Jim Knicely's very excellent blog site,
<https://yugabytedb.tips/>

This document is located here,

<https://www.yugabitten.com/62%20-%20Monthly%20articles/2025-12%20-%20MongoDB%20Secondary%20Indexes%20MongoDB%20secondary%20indexes%20from%20scratch.pdf>